

CAHIER DES CHARGES

Systeme de Gestion d'Examens

Exam Builder Platform

Projet	Exam Builder - Plateforme de creation d'examens academiques
Version	1.0
Date	2025
Technologies	React 18 + Node.js + Express + MySQL 8.0
Auteur	Equipe de developpement
Statut	En developpement actif

1. Presentation du Projet

1.1 Contexte

La creation d'examens academiques est une tache chronophage pour les enseignants. Elle implique la mise en page manuelle, la saisie repetitive de questions similaires, et la gestion difficile de plusieurs versions d'un meme document. L'application Exam Builder repond a ce besoin en offrant une plateforme web intuitive permettant aux enseignants de composer, previsualiser et exporter leurs examens en PDF en quelques minutes, sans competences techniques particulieres.

1.2 Objectifs

- Permettre la creation d'examens structures avec divers types de composants (QCM, vrai/faux, tableaux, zones de redaction, geometrie, timeline, matching...).
- Offrir une previsualisation en temps reel et un export PDF fidelement rendu.
- Gerer une banque de questions reutilisables entre plusieurs examens.
- Proposer des templates d'en-tetes pour standardiser les documents academiques.
- Securiser l'accès par un systeme d'authentification JWT robuste.
- Supporter trois langues : francais, anglais et arabe (RTL).

1.3 Perimetre

L'application est une solution web single-page (SPA) avec un backend RESTful autonome (aucune dependance a des services tiers payants comme Supabase). Elle cible les etablissements d'enseignement superieur et secondaire. La gestion des corrections automatiques et la passation d'examens en ligne sont hors perimetre pour la version 1.0.

2. Analyse des Besoins

2.1 Acteurs du Systeme

Enseignant	Acteur principal : cree, modifie, previsualise et exporte les examens.
Administrateur	Gere les comptes utilisateurs et les templates publics.
Base de donnees MySQL	Persistence de toutes les donnees (users, exams, questions, templates).
Generateur PDF	Produit le document final via jsPDF et html2canvas.

2.2 Besoins Fonctionnels - Tableau des Cas d'Utilisation

ID	Nom du Cas d'Utilisation	Acteur(s)	Description
UC-01	S'inscrire / Se connecter	Enseignant, Admin	Authentification via email + mot de passe, generation d'un token JWT.
UC-02	Creer un examen	Enseignant	Creer un nouvel examen avec titre et ajouter des composants.
UC-03	Ajouter des composants	Enseignant	Inserer QCM, Vrai/Faux, Tableau, Image, Zone redaction, Timeline, Matching...
UC-04	Reordonner les composants	Enseignant	Glisser-deposer (DnD) pour changer l'ordre des composants.
UC-05	Previsualiser l'examen	Enseignant	Afficher un rendu fidelement de l'examen dans le navigateur.
UC-06	Exporter en PDF	Enseignant	Generer et telecharger un fichier PDF avec options (filigrane, numerotation).
UC-07	Gerer la banque de questions	Enseignant	Sauvegarder, rechercher, filtrer et reutiliser des questions.
UC-08	Gerer les templates	Enseignant, Admin	Creer et appliquer des modeles d'en-tetes standardises.
UC-09	Supprimer un examen	Enseignant	Supprimer definitivement un examen de la base de donnees.
UC-10	Changer la langue	Enseignant	Basculer l'interface en FR / EN / AR (support RTL).

2.3 Besoins Non Fonctionnels

- Performance : chargement initial inferieur a 3 secondes, sauvegarde automatique toutes les 30 secondes.
- Securite : mots de passe haches avec bcryptjs, tokens JWT signes, protection CORS configuree.
- Compatibilite : navigateurs modernes (Chrome, Firefox, Edge, Safari) sans plugin requis.
- Maintenabilite : code TypeScript type, architecture MVC cote serveur, modules separes.
- Accessibilite : support complet RTL pour l'arabe, contrastes suffisants selon WCAG.

3. Architecture Technique

3.1 Stack Technologique

Frontend	React 18 + TypeScript + Vite + Tailwind CSS + Zustand (state management)
Backend	Node.js + Express + TypeScript + ts-node-dev (hot reload)
Base de donnees	MySQL 8.0 (tables : users, exams, question_bank, templates)
Authentification	JWT (jsonwebtoken) + bcryptjs (hachage mots de passe)
Generation PDF	jsPDF + html2canvas + KaTeX (rendu equations LaTeX)
Internationalisation	i18next (langues : Francais / Anglais / Arabe)
Drag and Drop	@dnd-kit/core + @dnd-kit/sortable
Requetes HTTP	Axios avec intercepteurs JWT automatiques

3.2 Structure du Projet

- server/src/routes/ - Points d'entree API : auth, exams, question-bank, templates
- server/src/middleware/ - Middleware d'authentification JWT
- server/src/database/ - Connexion MySQL et initialisation des tables
- client/src/pages/ - LoginPage, RegisterPage, DashboardPage, ExamBuilderPage, TemplatesPage, QuestionBankPage
- client/src/components/exam/ - Editeurs de composants (QCM, Tableau, Vrai/Faux, Image...)
- client/src/store/ - authStore, examStore (Zustand)
- client/src/utils/ - pdfGenerator.ts (generation et rendu PDF)
- client/src/locales/ - fr.json, en.json, ar.json (traductions)

3.3 Modele de Base de Donnees

users	id CHAR(36) PK, email UNIQUE, password_hash, first_name, last_name, institution, created_at
exams	id CHAR(36) PK, user_id FK, title, components LONGTEXT JSON, total_points, tags LONGTEXT JSON, updated_at
question_bank	id CHAR(36) PK, user_id FK, component LONGTEXT JSON, tags LONGTEXT JSON, difficulty, subject, usage_count
templates	id CHAR(36) PK, user_id FK, name, header_component LONGTEXT JSON, is_public TINYINT, usage_count

4. Catalogue des Composants d'Examen

4.1 Types de Composants Disponibles

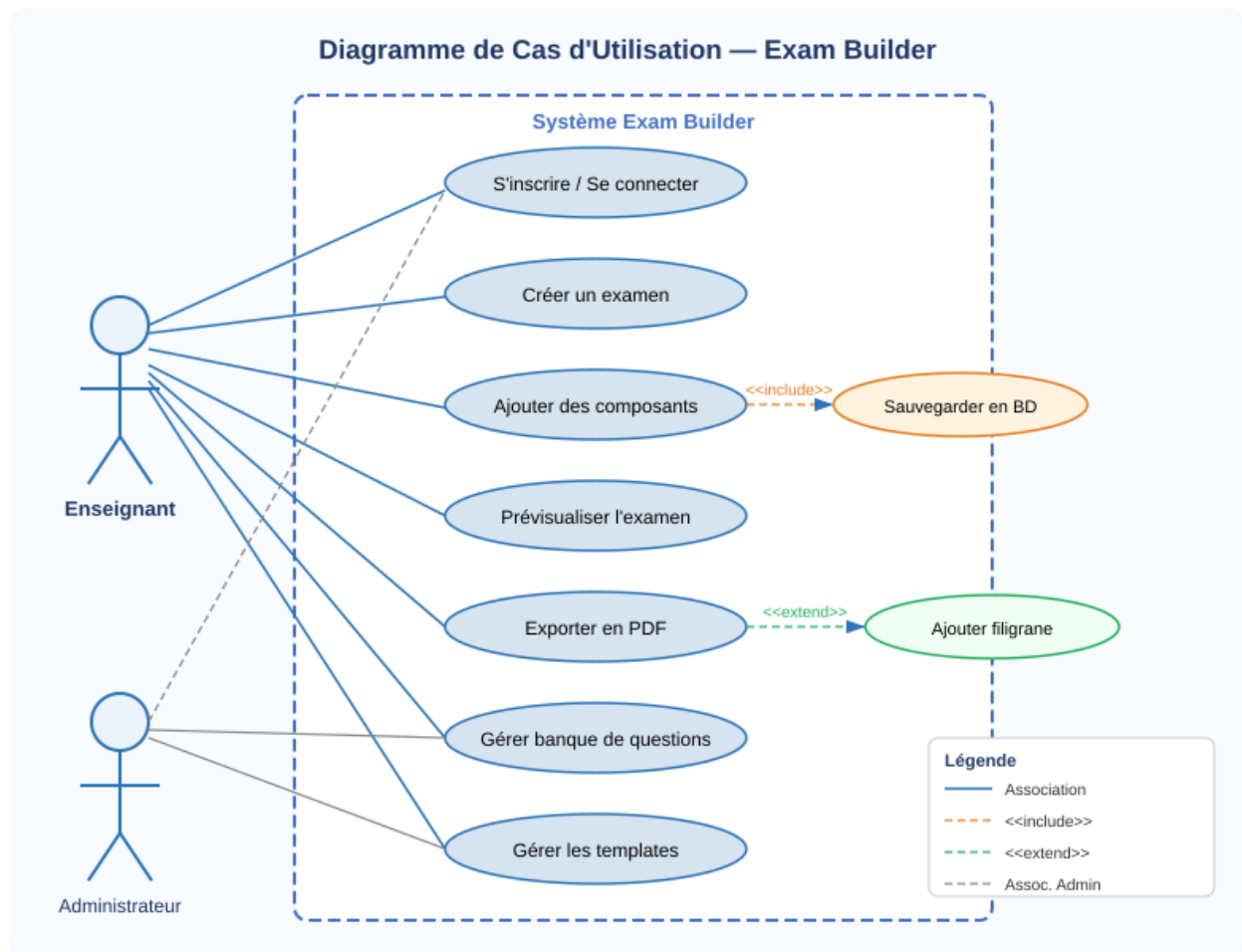
Composant	Description et parametres
En-tete (Header)	Titre de l'examen, annee academique, semestre, duree, logo uploadable, champs etudiant.
Texte / Question	Paragraphe libre avec support LaTeX, points optionnels, numérotation automatique.
QCM	Question a choix multiples, 2 a 6 options, mode 1 ou 2 colonnes, support LaTeX complet.
Vrai ou Faux	Tableau d'affirmations avec colonnes V/F (style cercles ou lettres configurable).
Tableau	Grille N lignes x M colonnes avec en-tetes editables et cellules libres.
Zone de redaction	Lignes reglees ou quadrillage, nombre de lignes configurable.
Image	Import d'image avec legende et redimensionnement maximal.
Compléter les blancs	Texte a trous avec balises [reponse] transformees en lignes de saisie.
Geometrie	Zone SVG avec grille millimetre / carreaux / points / isometrique.
En-tete Exercice	Bandeau bleu numerote avec total de points par exercice.
Timeline	Frise chronologique SVG avec dates et etiquettes alternees.
Matching	Tableau de correspondance colonnes A et B avec melange optionnel de la colonne B.
Saut de page	Separateur de page pour controler la pagination dans le PDF exporte.

5. Modelisation UML

5.1 Diagramme de Cas d'Utilisation

Le diagramme ci-dessous présente les interactions entre les acteurs (Enseignant, Administrateur) et les fonctionnalités principales du système. Les relations <<include>> et <<extend>> enrichissent les cas d'utilisation principaux.

Figure 1 - Diagramme de Cas d'Utilisation

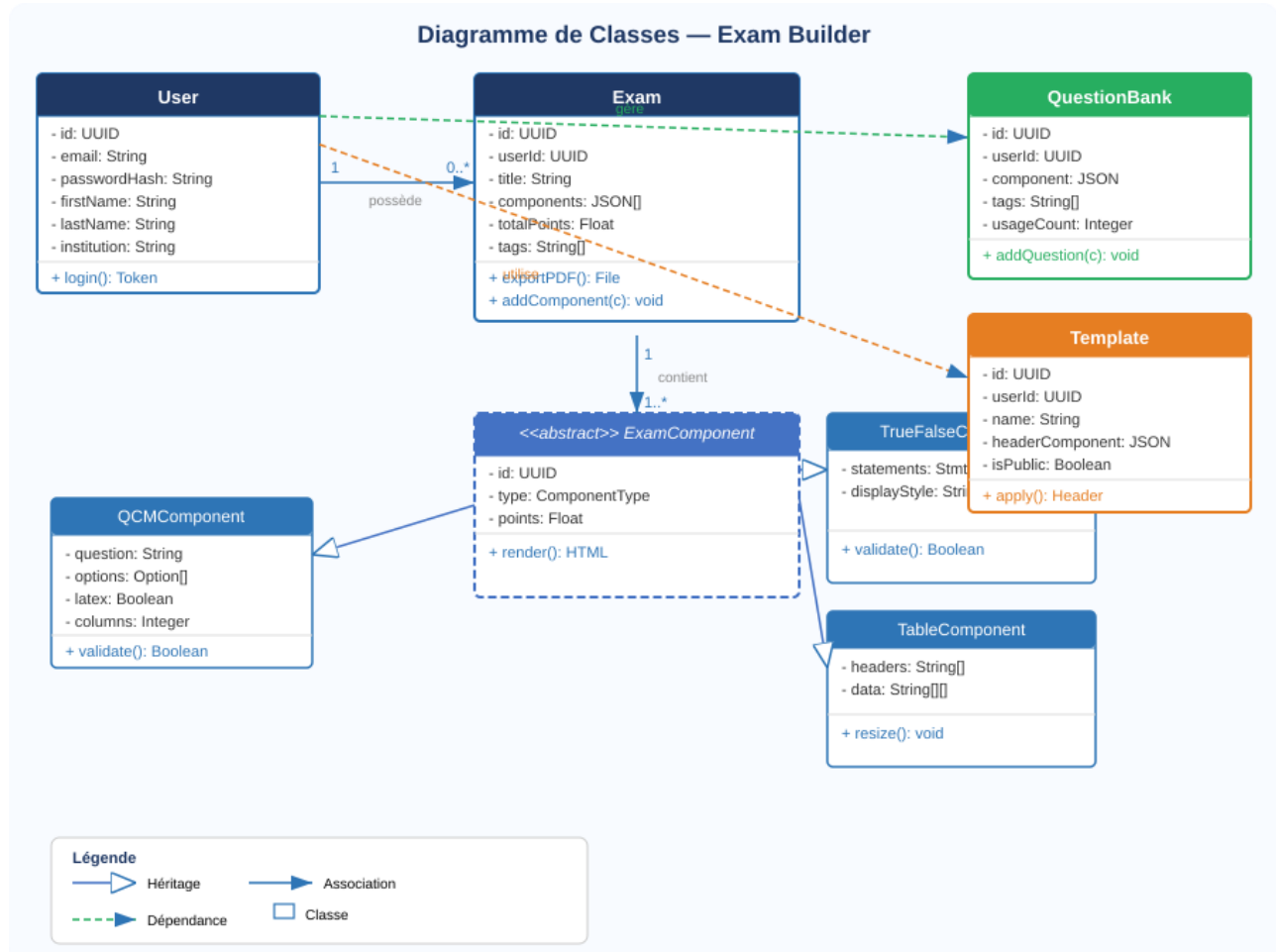


Les acteurs interagissent avec le système via des cas reliés par des associations, inclusions et extensions.

5.2 Diagramme de Classes

Le diagramme de classes modélise la structure statique du système : entités persistées en base de données, leurs attributs, méthodes principales et relations (association, héritage, dépendance).

Figure 2 - Diagramme de Classes

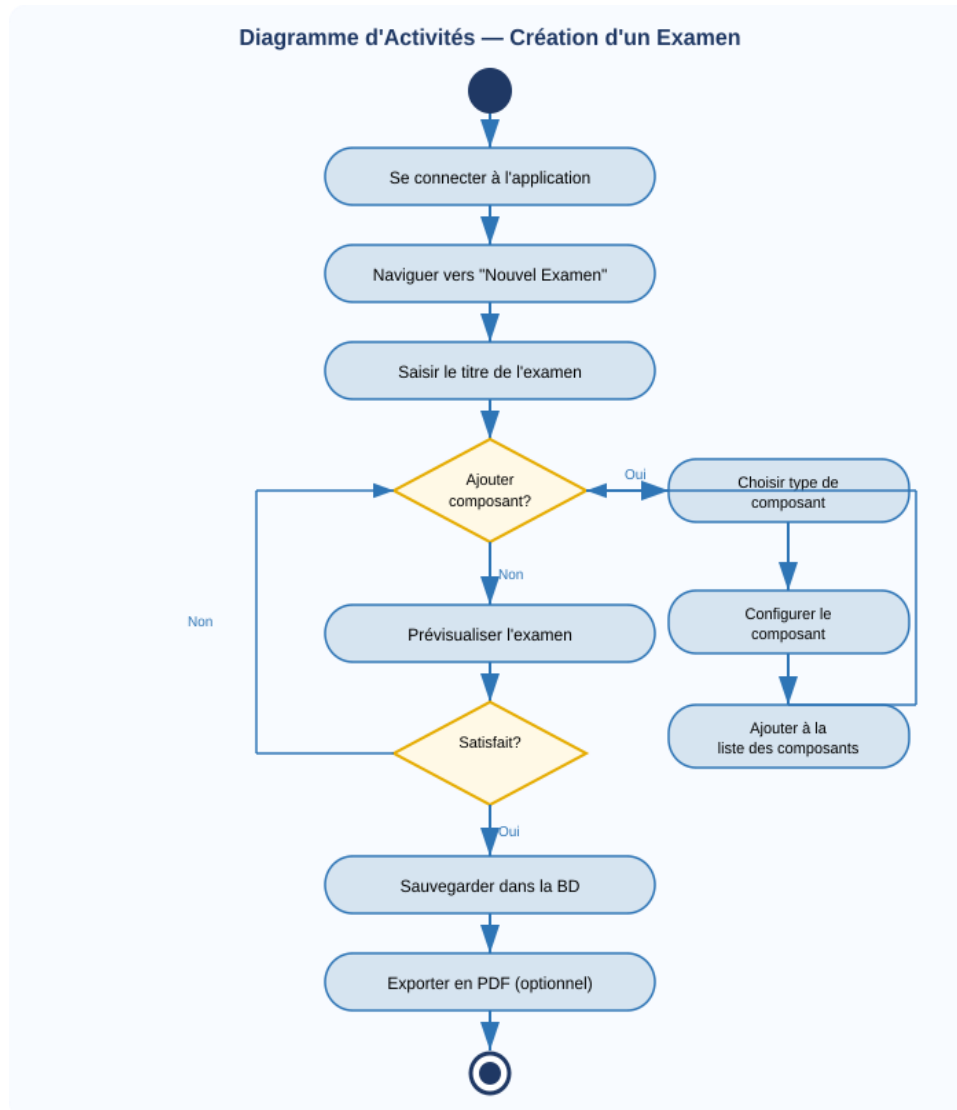


ExamComponent est une classe abstraite dont héritent QCMComponent, TrueFalseComponent, TableComponent, etc.

5.3 Diagramme d'Activites - Creation d'un Examen

Ce diagramme décrit le flux complet d'activités lorsqu'un enseignant crée et finalise un examen, depuis la connexion jusqu'à l'export PDF. Il inclut des nœuds de décision (ajouter composant, satisfait ?) et des boucles.

Figure 3 - Diagramme d'Activites

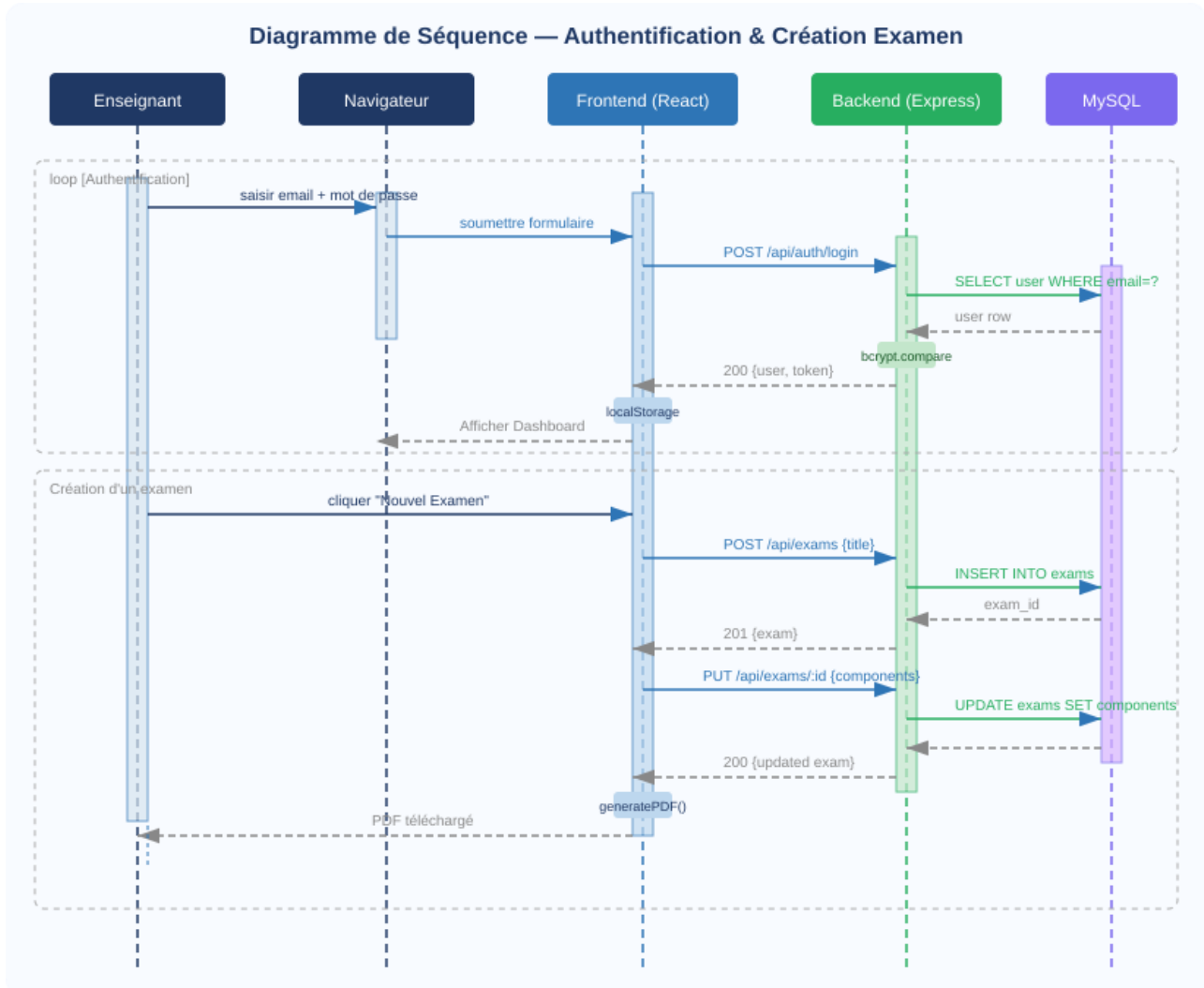


La boucle d'ajout de composants permet d'enrichir l'examen autant que nécessaire avant la sauvegarde finale.

5.4 Diagramme de Sequences - Authentification et Creation

Le diagramme de sequences illustre les echanges chronologiques entre l'enseignant, le frontend React, l'API Express et la base de donnees MySQL lors de deux scenarios : la connexion (JWT) et la creation d'un examen.

Figure 4 - Diagramme de Sequences

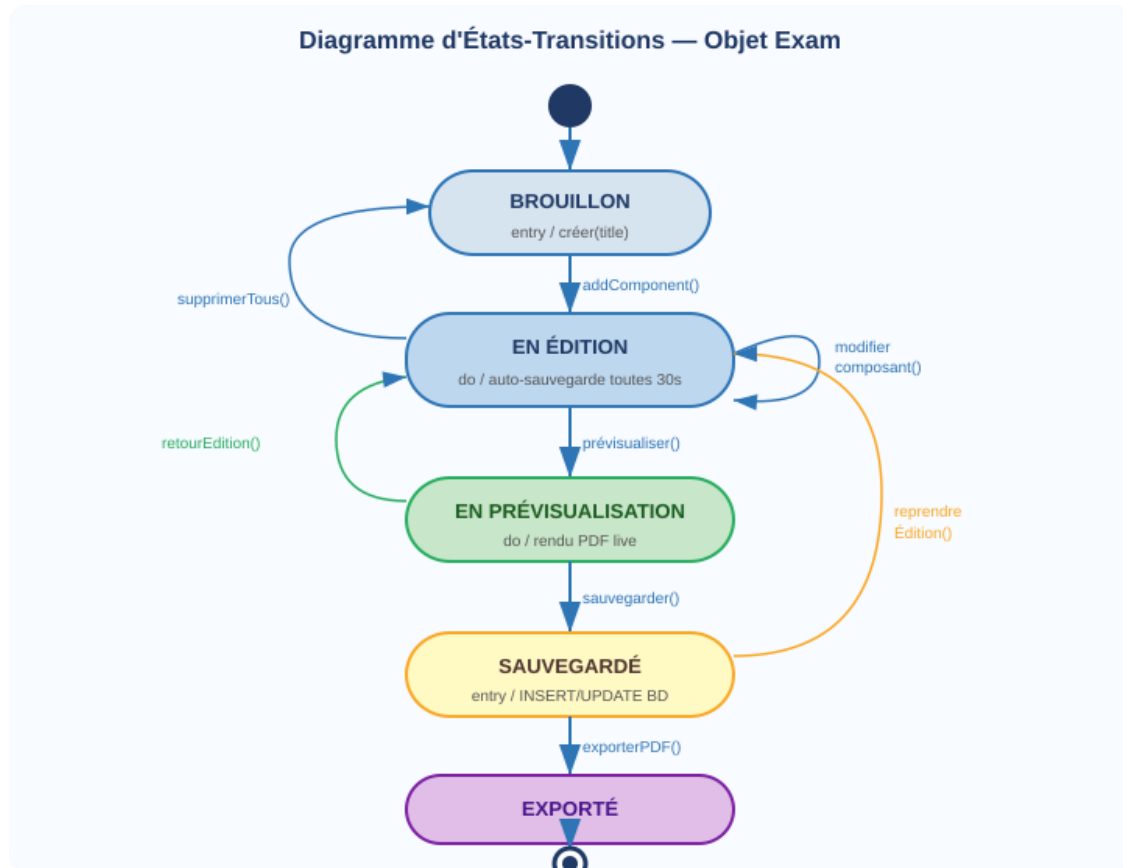


Les tirets representent les messages de retour. Les boites sur les lignes de vie indiquent les periodes d'activation.

5.5 Diagramme d'Etats-Transitions - Objet Exam

L'objet Exam est l'objet significatif central du systeme. Son cycle de vie est modelise ci-dessous : il evolue de l'etat BROUILLON a EN EDITION, puis PREVISUALISATION, SAUVEGARDE et enfin EXPORTE. Des transitions permettent de revenir en edition a tout moment.

Figure 5 - Diagramme d'Etats-Transitions de l'objet Exam



L'objet Exam peut retourner en edition depuis l'etat SAUVEGARDE grace a la transition reprendreEdition().

6. Planning et Livrables

6.1 Phases de Developpement

Phase	Contenu	Livable
Phase 1 - Initialisation	Architecture, configuration MySQL, squelette React + Express.	Projet fonctionnel vide
Phase 2 - Authentification	Inscription, connexion, middleware JWT, formulaires React.	Auth complete
Phase 3 - Editeur	Tous les editeurs de composants, drag and drop, previsualisation.	Editeur fonctionnel
Phase 4 - PDF et Export	pdfGenerator, rendu LaTeX, filigrane, grille correction.	Export PDF operationnel
Phase 5 - Banque et Templates	CRUD banque de questions, templates publics et privs.	Modules avances
Phase 6 - i18n et UX	Internationalisation FR/EN/AR, theme sombre, responsive.	Interface polie
Phase 7 - Tests et Deploiement	Tests, optimisations, documentation technique finale.	Version 1.0 livree

6.2 Contraintes Techniques

- Aucune dependance a Supabase ou service cloud payant : tout est auto-heberge.
- Base de donnees MySQL 8.0 obligatoire (port configurable, ex: 3307).
- Node.js version 18+ requis pour crypto.randomUUID() natif.
- Le frontend Vite proxie automatiquement /api vers le backend Express (port 5000).

7. Conclusion

Le projet Exam Builder constitue une solution web complete et autonome pour la creation d'examens academiques. Son architecture decouplée (frontend React / backend Express / base MySQL) garantit la maintenabilite et l'evolutivite sans dependance a des services tiers.

Les diagrammes UML presentes dans ce cahier des charges definissent clairement : les interactions entre acteurs (cas d'utilisation), la structure des donnees (classes), les flux comportementaux (activites, sequences) et le cycle de vie de l'objet central Exam (etats-transitions).

Les versions futures pourront integrer : la correction automatique des QCM, la passation d'examens en ligne par les etudiants, la generation de questions par intelligence artificielle, et le partage collaboratif entre enseignants.